AeroNode

Adaptive Edge-AI Inference Runtime — Engineering Project Brief

Prepared for an AI implementation agent. This document is the authoritative specification. Treat every constraint and phase boundary as binding unless it conflicts with the actually installed toolchain, in which case document the deviation and proceed with the technically correct choice.

Primary stack	C++17 · ONNX Runtime (C++ API) · PyTorch · Python (simulator) · FastAPI · PostgreSQL · CMake · Zero

---

1 · Project Identity

One-line definition: AeroNode is a software-defined edge-AI runtime that dynamically controls neural-network inference depth according to real-time resource constraints and latency requirements, using multi-exit neural networks and confidence-based early termination.

2 · Problem Statement

Why: Traditional edge-AI inference usually executes a fixed computational path regardless of current CPU load, temperature, or latency requirements. Edge resources are highly variable, so inference should be able to adapt.

Concept: Instead of always executing the full model, AeroNode places intermediate classifier heads throughout a neural-network backbone. When an intermediate prediction is sufficiently reliable and the current runtime policy permits it, inference terminates early.

3 · Core Engineering Question

Research question: Can adaptive early-exit inference reduce latency and computational workload on edge-class hardware while maintaining acceptable prediction accuracy under changing runtime constraints?

Success criterion: The project must demonstrate measurable behavior using benchmarks, not merely implement an early-exit model. It must prove the ability to handle distributed edge topologies under simulated duress.

4 · High-Level Architecture

Data flow:

```
Python Control Plane (CPU temp / latency constraints)
| ZeroMQ (IPC channel)
v
Adaptive C++ Runtime --(loads/executes)--> ONNX Runtime + multi-exit model
|
v
FastAPI telemetry / dashboard --> PostgreSQL
```

ASE abstraction:

- **Environment:** Python simulator generating resource conditions.

- **Agent:** Adaptive runtime policy deciding inference depth.
- **System:** C++ runtime executing the neural network and measuring performance.

5 · Neural Network & Dataset Design

- **Dataset:** Imagenette (10-class subset of ImageNet) — rapid local training with realistic, high-resolution imagery.
- **Initial model:** A modest ResNet-style backbone, not a large production model.
- **Training:** Train the backbone with intermediate classifier heads using a weighted multi-exit loss. Verify that intermediate exits produce useful predictions before integrating the C++ runtime.

Multi-exit structure:

```
Input -> Stem -> Block 1 -> (Exit 1)
-> Block 2 -> (Exit 2)
-> Block 3 -> (Exit 3)
-> Block 4 -> (Final Exit)
```

6 · Confidence / Entropy

Formula (Shannon entropy):

```
H(X) = - sum_i [ p_i * log(p_i) ]

where p_i = predicted probability for class i
```

Interpretation: Lower entropy generally indicates a more concentrated probability distribution; higher entropy indicates greater uncertainty.

Initial policy:

```
entropy = calculate_entropy(probabilities)
if entropy <= threshold:
EXIT
else:
CONTINUE
```

Constraint: The threshold/policy must be experimentally validated. Do not assume a particular threshold-to-latency relationship without benchmarking.

7 · Runtime Architecture & Phasing

Core responsibilities: The C++17 runtime loads the model, initializes ONNX Runtime, executes inference stages, obtains intermediate predictions, calculates entropy, consults the runtime policy, decides EXIT/CONTINUE, records metrics, and returns the prediction.

Implementation phasing:

- **MVP phase:** Extract the intermediate tensor back to C++ memory using standard Session::Run outputs, and calculate entropy using standard C++ array math.
- **Advanced phase:** Shift the entropy calculation directly into the ONNX execution provider by writing an Ort::CustomOpDomain, achieving zero-overhead graph mutation.

Constraint: Do not assume a standard ONNX Runtime session can arbitrarily pause and resume inside one optimized graph. Do not make dynamic graph mutation the initial core mechanism — represent stages as separate sessions/models or another architecture that is actually supported and benchmarkable.

8 · Control Plane & Inter-Process Communication (IPC)

Purpose: Python simulates changing edge conditions so the runtime can be evaluated without requiring dedicated hardware.

IPC mechanism: To avoid bottlenecking the C++ inference engine, the Python control plane and C++ runtime MUST communicate over a low-latency ZeroMQ (ZMQ) channel or local UDP sockets.

Signals: CPU utilization, simulated temperature, latency budget, compute/resource pressure.

Feedback loop:

```
Environment -> telemetry -> runtime policy -> inference behavior -> measurements -> Environment
```

Policy: Map environmental constraints to an allowable inference budget, then combine that budget with entropy/confidence to choose whether to exit or continue.

9 · Memory / Allocator Experiments (Hardening)

Phase: Optimization and experimental phase only — not part of the initial MVP.

Method: Compare ONNX Runtime default allocation behavior against controlled allocator/arena configurations supported by the installed ONNX Runtime version. Override the extension strategy to `kSameAsRequested` via the C++ API. Use `CreateAndRegisterAllocator` to share a single memory arena across the process.

Metrics: Peak memory, allocation behavior, latency, repeatability, and measurable fragmentation indicators.

Constraint: Any performance or deterministic-memory claim must be strictly supported by measurements. Never claim an allocator strategy automatically guarantees deterministic memory limits or eliminates fragmentation.

10 · Telemetry / FastAPI

Purpose: FastAPI and PostgreSQL form a supporting telemetry layer, not the core of the project.

Expose: CPU load, temperature, latency budget, current entropy, selected exit, inference latency, prediction, confidence, estimated compute saved, and deadline status.

Constraint: Build the dashboard only after the C++ runtime and benchmark pipeline are functional.

11 · Benchmarking Plan

Baselines:

- Full model — always execute to final exit.
- Static early exit — fixed threshold.
- Adaptive AeroNode policy.

Metrics: Accuracy, average latency, P95 latency, CPU usage, peak memory, exit distribution, compute saved, and deadline violations.

Goal: Show when and why AeroNode selects different exits, and quantify the accuracy/latency/resource trade-offs.

12 · Fundamental Trade-off

```
Accuracy <-> Latency <-> Compute <-> Resource Constraints
```

Aggressive early exits can reduce latency and computation while increasing the risk of incorrect predictions. Conservative policies preserve accuracy at the cost of additional computation and latency.

13 · Development Phases

Phase	Focus	Deliverable
0	Architecture	architecture.md, current_status.md, README.md — interfaces, assumptions, constraints, decisions
1	ML prototype	Build and train a multi-exit model (Imagenette); validate intermediate exits
2	ONNX export	Export model to ONNX; inspect with Netron
3	C++ runtime	C++17 + CMake + ONNX Runtime; achieve verified basic inference
4	IPC & early exit	Implement ZeroMQ channel; intermediate outputs → entropy → exit/continue
5	Adaptive policy	CPU/temperature/latency constraints → runtime decision
6	Benchmarking	Compare full inference, static early exit, adaptive AeroNode
7	Memory experiments	Allocator/arena configurations (kSameAsRequested) and measurements
8	Telemetry	PostgreSQL, FastAPI, and visualization
9	Documentation	Architecture, experiments, benchmark results, failures, trade-offs, limitations, future work

14 · Repository Structure

```
AeroNode/
■■■ README.md
■■■ architecture.md
■■■ current_status.md
■■■ CMakeLists.txt
■■■ .gitignore
■■■ model/
■ ■■■ training/
■ ■■■ export/
■ ■■■ artifacts/
■■■ runtime/
■ ■■■ include/
■ ■■■ src/
■ ■■■ CMakeLists.txt
■■■ control_plane/
■ ■■■ simulator.py
■ ■■■ policy.py
■ ■■■ ipc_zmq/
■■■ api/
■ ■■■ main.py
■ ■■■ database/
■■■ benchmarks/
■ ■■■ results/
■ ■■■ scripts/
■■■ tests/
■■■ docs/
```

Constraint: Do not create every file immediately. Start with the minimum required vertical slice and expand only when justified.

15 · Build System

Recommendation: Use CMake for the C++17 + ONNX Runtime project. Initially target the available Windows/MSVC environment if that is where development is happening; preserve portability to Ubuntu/WSL2 later.

Reason: CMake provides reproducible dependency configuration, build targets, and cross-platform support without manually maintaining Makefiles.

16 · Engineering Standards

Avoid:

- Over-engineering the MVP; unnecessary frameworks.
- Fake claims about deterministic performance or energy savings without measurements.
- Making graph mutation mandatory.
- Building the dashboard before the runtime.
- Copying APIs without checking installed-version compatibility.
- Using Python for core inference — the core inference path must be C++.

Do:

- Verify ONNX Runtime APIs against the installed version.
- Benchmark performance; keep components modular; use CMake.
- Test entropy/policy logic; document assumptions.
- Maintain a baseline; use reproducible benchmark scripts.
- Commit logical milestones.

17 · MVP Definition

```
PyTorch multi-exit model
-> ONNX
-> C++17 inference
-> intermediate prediction
-> entropy calculation
-> early exit
-> ZeroMQ communication
-> changing simulated constraints
-> adaptive exit policy
-> latency / accuracy measurements
```

Not the MVP: A production-grade edge-AI operating system, a complete dynamic graph-rewriting engine, or a fully optimized memory runtime.

18 · Portfolio / Engineering Value

Focus: The project combines machine learning, systems programming, runtime optimization, edge computing, resource-aware scheduling, and performance engineering.

Demonstration: The strongest result is not the number of features. It is a reproducible demonstration of how runtime constraints change inference behavior, and what accuracy/latency/resource trade-off results.

19 · Final Demonstration

```
Input
-> Multi-Exit Neural Network
Exit 1 (~8ms) | Exit 2 (~15ms) | Exit 3 (~24ms) | Final (~38ms)
-> Runtime constraints (CPU / thermal / latency SLA)
-> Adaptive policy
-> Selected exit + measured metrics
```

Success: AeroNode should explain and quantify why a particular exit was selected under a given runtime state.

20 · Instructions for the Implementation Agent

Directive: You are the primary engineer responsible for implementing AeroNode. Read this brief completely. Treat it as the authoritative project specification. First inspect the development environment and determine the available OS, compiler, CMake, Python, PyTorch, ONNX, ONNX Runtime, VS Code configuration, and Git configuration.

Documentation first: Before substantial implementation, create `architecture.md` and `current_status.md`. `architecture.md` must document actual architecture, component boundaries, data flow, interfaces, assumptions, constraints, and technology choices. `current_status.md` must contain Completed, In Progress, Blocked, Next Step, Known Issues, and Technical Decisions.

Implementation order: Start with the smallest working vertical slice: simple model → ONNX → C++ inference → verified output. Then progressively add multi-exit → entropy → early exit → ZeroMQ IPC → adaptive policy → benchmarks → memory experiments → telemetry.

Decision-making: Do not blindly implement this blueprint. Verify APIs and execution semantics against the installed ONNX Runtime version. If the proposed design conflicts with actual runtime capabilities, choose the technically correct architecture and document the deviation.

Autonomy: Do not ask for permission for every small implementation decision. Make reasonable engineering decisions, document them, and continue. Ask only when a genuine blocker requires information unavailable from the environment.

Quality bar: Prioritize measurable engineering results over feature count. Preserve reproducibility, tests, benchmarks, clear interfaces, and documentation.